

PRESENTACIÓN DE

# PROYECTO

RUNWELL

- Integrantes:
- Aly Rodriguez
- Anyer Silva
- Wilmer Rojas

- Introducción
- Objetivos
- Arquitectura del sistema
- Modulos
- Tecnologias
- Funcionalidades principales
- Decisiones
- Pruebas
- Despedida

# ÍNDICE

RunWell es una aplicación web de seguimiento de running construida con HTML, CSS y JavaScript (módulos ES, sin frameworks ni backend). Usa Leaflet + OpenStreetMap para mapas interactivos y un objeto de estado compartido que conecta los módulos de geolocalización, actividad, proximidad e historial.

El sistema se basa en geolocalización con watchPosition, cálculo de distancias con Haversine y persistencia en localStorage. Incluye detección de proximidad por colisión punto-segmento (50 m), panel HUD en tiempo real con cronómetro y ritmo, y despliegue en Vercel con Supabase para logs y datos.

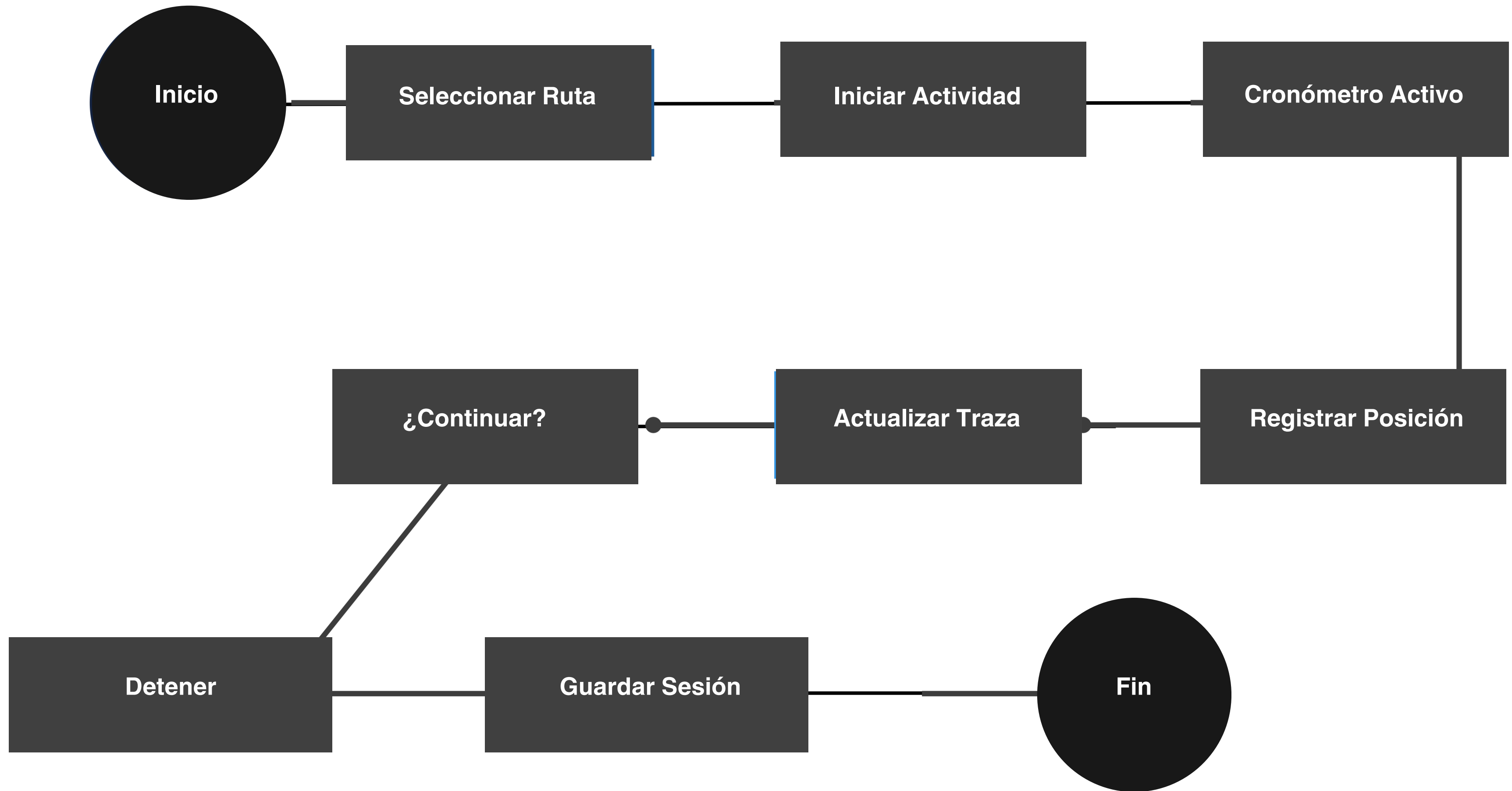
# INTRODUCCIÓN

# OBJETIVOS

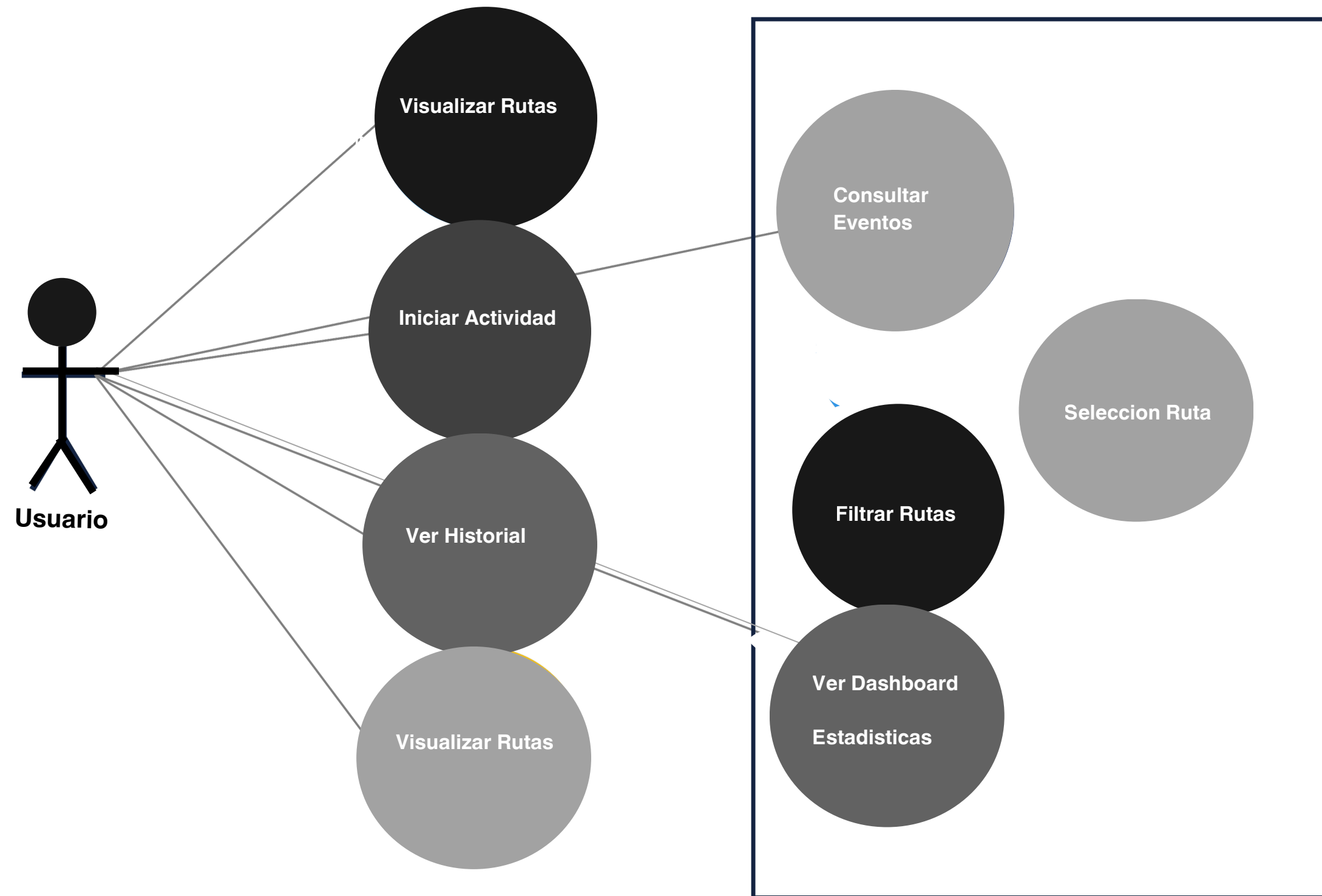
**Diseñar e implementar una aplicación web de seguimiento de running que permita al usuario visualizar rutas, registrar actividades y consultar su historial de sesiones.**

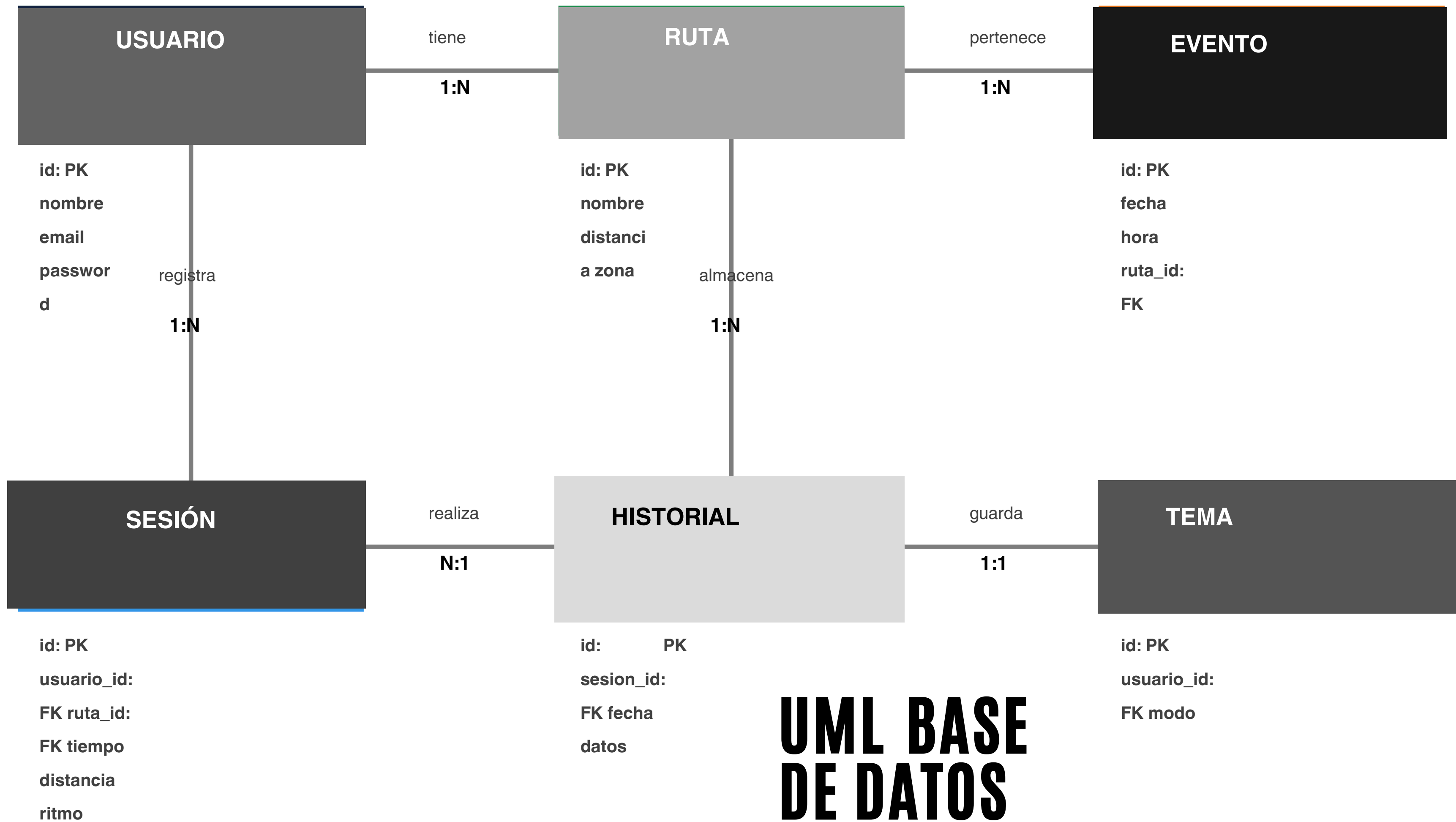
- Visualizar rutas predefinidas sobre un mapa interactivo**
- Iniciar y detener actividades de trote o carrera con cronómetro**
- Calcular distancia recorrida y ritmo en tiempo real**
- Almacenar el historial de sesiones de forma persistente**
- Implementar geolocalización con seguimiento continuo**
- Detectar la proximidad del usuario a una ruta y sugerir inicio de actividad**

# UML DE ACTIVIDADES



# UML DE ACTORES





**UML BASE  
DE DATOS**

# ARQUITECTURA DEL SISTEMA

RunWell sigue una arquitectura de frontend puro, compuesta por tres capas:

- Capa de presentación: HTML + CSS con diseño responsivo y modo oscuro
- Capa de lógica: JavaScript con módulos ES, organizados en archivos de responsabilidad única
- Capa de datos: localStorage para persistencia local y Supabase (BaaS) para logs y datos en la nube

La aplicación no requiere servidor ni framework de build: se sirve como archivos estáticos y se despliega en Vercel. El estado central se gestiona en un objeto compartido (estado) que actúa como punto de comunicación entre módulos.



**El código se organiza en módulos ES con un grafo de dependencias acíclico:**

- datos.js — rutas predefinidas (mock data)**
- util.js — funciones puras: tiempo, distancia (Haversine), ritmo, colisión**
- estado.js — objeto compartido con el estado global**
- dom.js — referencias a elementos del DOM**
- historial.js — guardado, lectura y migración en localStorage**
- actividad.js — cronómetro, HUD, traza y guardado de sesiones**
- proximidad.js — detección de cercanía a rutas**
- geolocalizacion.js — seguimiento GPS y marcador del usuario**
- mapa.js — apertura/cierre del mapa, dibujo de rutas y popups**
- eventos.js — tarjetas de la sección Eventos**
- navegacion.js — mostrar/ocultar secciones**
- tema.js — modo oscuro persistente**
- main.js — punto de entrada que conecta botones con módulos**

# **MODULOS**



# TECNOLOGIAS

**HTML5 — estructura semántica de la página**

**CSS3 — diseño responsivo, modo oscuro, animaciones**

**JavaScript (ES6+) — lógica de la aplicación con módulos ES**

**Leaflet.js — librería de mapas interactivos**

**OpenStreetMap — tiles de mapas**

**Geolocation API — seguimiento de ubicación del usuario**

**localStorage — persistencia de sesiones y preferencias**

**Supabase — BaaS para logs y almacenamiento**

**Vercel — despliegue y hosting**

# **FUNCIONALIDADES PRINCIPALES**

**Mapa interactivo: visualización de rutas predefinidas en Maracaibo con marcadores de inicio y fin**

**Panel HUD en vivo: cronómetro, distancia y ritmo actualizados en tiempo real durante la actividad**

**Traza del recorrido: línea punteada que crece con cada posición GPS registrada**

**Historial de sesiones: listado de actividades previas con fecha, tiempo, distancia y ritmo**

**Detección de proximidad: al acercarse a una ruta, la app sugiere iniciar una actividad**

**Modo oscuro: tema persistente almacenado en localStorage**



# DECISIONES

**Leaflet + OpenStreetMap:** librería liviana, open source y sin costos de API keys, ideal para un proyecto académico

**Módulos ES sin build step:** organización del código en archivos separados sin necesidad de bundler, facilitando la depuración y el mantenimiento

**localStorage para persistencia:** permite guardar sesiones sin backend, con migración automática de formatos legacy para no perder datos del usuario

**Manejo de errores de geolocalización:** se aplica la regla "lo automático es silencio, lo que pide el usuario avisa" — las peticiones automáticas no muestran alertas para no interrumpir la experiencia, mientras que las acciones manuales (botón ) informan el estado del GPS

**Descarte de lecturas imprecisas:** lecturas con precisión mayor a 35 metros se descartan para garantizar la calidad del tracking

**La validación se realizó mediante arneses temporales que ejercitan la aplicación con clics reales en Firefox headless, cubriendo cuatro escenarios de geolocalización: permiso pendiente, concedido, denegado y error.**

**Cada escenario se ejecutó con un perfil de Firefox desechable para garantizar aislamiento entre pruebas.**

## **Resultados:**

- 23/23 checks de geolocalización superados**
- 23/23 checks de regresión superados (mapa, popup, actividad → detener → historial, tema, carga con permiso denegado)**
- 0 errores de consola**

# **PRUEBAS**

**GRACIAS  
POR VER!**

**RUNWELL**